

Playwright for Python

Complete Concepts & Examples Guide

Browser Automation | Web Testing | Scraping

Table of Contents

1. Installation & Setup
2. Browser & Context Management
3. Page Navigation
4. Selectors & Locators
5. User Interactions
6. Waiting Strategies
7. Assertions
8. Forms & File Uploads
9. Screenshots & PDFs
10. Network Interception
11. JavaScript Execution
12. Frames & iFrames
13. Dialogs & Popups
14. Authentication
15. Async API
16. pytest Integration
17. Page Object Model
18. Debugging & Tracing
19. Mobile Emulation
20. Best Practices

1. Installation & Setup

Playwright is a browser automation library that supports Chromium, Firefox, and WebKit. Install it via pip and then download the browser binaries.

Installation

```
# Install the Python package
pip install playwright

# Download browser binaries (Chromium, Firefox, WebKit)
playwright install

# Install only specific browser
playwright install chromium

# Install with system dependencies (Linux)
playwright install --with-deps
```

Verify Installation

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.goto('https://playwright.dev')
    print(page.title()) # Playwright
    browser.close()
```

■■ **Note:** Playwright supports Python 3.8+. Always close browsers to avoid resource leaks.

Browser Types

Browser	Engine	Use Case
p.chromium	Blink (Chrome/Edge)	Default; best compatibility
p.firefox	Gecko	Firefox-specific testing
p.webkit	WebKit (Safari)	Safari/iOS simulation

2. Browser & Context Management

Playwright uses a three-tier hierarchy: **Browser** → **BrowserContext** → **Page**. Contexts are isolated sessions (like incognito windows). Each page lives inside a context.

Launching Browsers

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    # Headless (default) - no visible window
    browser = p.chromium.launch(headless=True)

    # Headed - visible browser window
    browser = p.chromium.launch(headless=False)

    # Slow motion - useful for debugging demos
    browser = p.chromium.launch(slow_mo=500)

    # Custom executable path
    browser = p.chromium.launch(
        executable_path='/usr/bin/google-chrome'
    )
```

Browser Context

```
# Create isolated context (like incognito)
context = browser.new_context(
    viewport={'width': 1280, 'height': 720},
    locale='en-US',
    timezone_id='America/New_York',
    geolocation={'latitude': 40.7128, 'longitude': -74.0060},
    permissions=['geolocation'],
    http_credentials={'username': 'user', 'password': 'pass'},
    record_video_dir='videos/', # record video
    record_har_path='trace.har', # record HAR
)

# Create a page within the context
page = context.new_page()

# Multiple independent contexts (different sessions)
ctx1 = browser.new_context()
ctx2 = browser.new_context() # completely separate session

# Always close context when done
context.close()
browser.close()
```

■ **Tip:** Use multiple BrowserContexts to simulate multiple logged-in users in the same test.

Persistent Context (Saved Login State)

```
# Persist cookies/localStorage across runs
context = p.chromium.launch_persistent_context(
    user_data_dir='/tmp/my_profile',
```

```
        headless=False  
    )  
    page = context.pages[0]
```

3. Page Navigation

```
# Navigate to URL
page.goto('https://example.com')

# With wait-until option
page.goto('https://example.com', wait_until='networkidle')
# Options: 'load', 'domcontentloaded', 'networkidle', 'commit'

# Browser history
page.go_back()
page.go_forward()
page.reload()

# Get current URL and title
print(page.url)
print(page.title())

# Get page content
html = page.content()

# Set page content directly
page.set_content('<html><body><h1>Hello</h1></body></html>')

# Wait for navigation to complete
with page.expect_navigation():
    page.click('a#link')    # triggers navigation

# Navigate and wait together
page.goto('https://example.com', timeout=30000)    # 30s timeout
```

wait_until	Description
load	Wait for 'load' event (default)
domcontentloaded	Wait for DOM to be parsed
networkidle	Wait until no network requests for 500ms
commit	Wait for first byte received

4. Selectors & Locators

Locators are the recommended way to find elements. They are auto-waiting, retry-able, and strict by default.

CSS Selectors

```
# By tag
page.locator('button')

# By class
page.locator('.btn-primary')

# By ID
page.locator('#submit-btn')

# Attribute
page.locator('[data-testid="login"]')
page.locator('input[type="email"]')

# Nested
page.locator('.form .input-group input')

# nth child
page.locator('ul > li:nth-child(2)')
```

Text Selectors

```
# Exact text
page.get_by_text('Sign In')

# Partial text (default)
page.get_by_text('Sign', exact=False)

# Inside locator
page.locator('button').filter(has_text='Submit')
```

Semantic / Role Selectors (Preferred)

```
# By ARIA role - most resilient to UI changes
page.get_by_role('button', name='Submit')
page.get_by_role('link', name='Home')
page.get_by_role('textbox', name='Email')
page.get_by_role('checkbox', name='Remember me')
page.get_by_role('heading', name='Dashboard')
page.get_by_role('combobox', name='Country')

# By label
page.get_by_label('Email address')

# By placeholder
page.get_by_placeholder('Enter your email')

# By alt text (images)
page.get_by_alt_text('Company logo')

# By title attribute
```

```
page.get_by_title('Close dialog')

# By test ID (data-testid attribute)
page.get_by_test_id('submit-form')
```

Chaining & Filtering Locators

```
# Filter by text
page.locator('li').filter(has_text='Apple')

# Filter by child element
page.locator('li').filter(has=page.locator('img'))

# Exclude
page.locator('li').filter(has_not_text='Banana')

# Narrow scope
sidebar = page.locator('#sidebar')
sidebar.get_by_role('link').first

# nth item
page.locator('tr').nth(2)          # 0-indexed
page.locator('tr').first
page.locator('tr').last

# XPath
page.locator('xpath=//button[@type="submit"]')
```

Locator Operations

```
loc = page.locator('.items li')

# Count matching elements
count = loc.count()

# Get all matching elements as list
items = loc.all()

# Get inner text
text = loc.inner_text()

# Get all texts
texts = loc.all_inner_texts()

# Get attribute
href = loc.get_attribute('href')

# Check visibility
visible = loc.is_visible()
enabled = loc.is_enabled()
checked = loc.is_checked()
```

■ **Tip:** Prefer `get_by_role`, `get_by_label`, and `get_by_test_id` — they are the most stable selectors.

5. User Interactions

Clicking

```
# Simple click
page.click('button#submit')
page.locator('button').click()

# Double click
page.dblclick('.cell')

# Right click
page.click('.item', button='right')

# Click with modifier keys
page.click('a', modifiers=['Shift'])      # Shift+Click
page.click('a', modifiers=['Control'])    # Ctrl+Click

# Click at specific coordinates
page.click('.canvas', position={'x': 100, 'y': 200})

# Force click (bypasses actionability checks)
page.click('.hidden-btn', force=True)

# Click with delay
page.click('button', delay=100)
```

Typing & Filling

```
# Fill input (clears first, then types)
page.fill('input[name="email"]', 'user@example.com')
page.get_by_label('Password').fill('secret123')

# Type character by character (triggers key events)
page.type('input', 'Hello World', delay=50)

# Clear an input
page.fill('input', '')      # fill with empty string
page.locator('input').clear()

# Press key
page.press('input', 'Enter')
page.press('body', 'Escape')
page.press('input', 'Control+a')  # select all
page.press('input', 'Control+c')  # copy

# Keyboard actions directly
page.keyboard.press('Tab')
page.keyboard.type('Hello')
page.keyboard.down('Shift')
page.keyboard.up('Shift')
```

Dropdowns / Select


```
# Select by value
page.select_option('select#country', 'us')

# Select by label (visible text)
page.select_option('select', label='United States')

# Select by index
page.select_option('select', index=2)

# Multi-select
page.select_option('select[multiple]', ['option1', 'option2'])

# Using locator
page.get_by_role('combobox').select_option('value')
```

Checkboxes & Radio Buttons

```
# Check a checkbox
page.check('input[type="checkbox"]')
page.locator('#terms').check()

# Uncheck
page.uncheck('#newsletter')

# Toggle (click-based)
page.click('input[type="checkbox"]')

# Radio button
page.check('input[value="male"]')

# Verify state
is_checked = page.is_checked('#accept')
```

Hover & Mouse

```
# Hover over element
page.hover('.dropdown-trigger')

# Mouse move
page.mouse.move(100, 200)

# Mouse click at coordinates
page.mouse.click(300, 400)

# Drag and drop
page.drag_and_drop('#source', '#target')

# Manual drag
page.mouse.move(100, 100)
page.mouse.down()
page.mouse.move(300, 300)
page.mouse.up()

# Scroll
page.mouse.wheel(0, 300)          # scroll down 300px
```

6. Waiting Strategies

Playwright has built-in auto-waiting for most actions. Explicit waits are only needed in special cases.

Auto-Waiting (Built-in)

Locator actions like `click()`, `fill()`, etc. automatically wait for the element to be: visible, stable, not obscured, enabled, and editable.

Explicit Waits

```
# Wait for element to appear
page.wait_for_selector('.result', state='visible')
page.wait_for_selector('.spinner', state='hidden')
page.wait_for_selector('.modal', state='attached')    # in DOM
page.wait_for_selector('.card', state='detached')    # removed

# Wait for locator
page.locator('.toast').wait_for(state='visible')

# Wait for page events
page.wait_for_load_state('networkidle')
page.wait_for_load_state('domcontentloaded')
page.wait_for_load_state('load')

# Wait for URL
page.wait_for_url('**/dashboard')
page.wait_for_url('https://example.com/home')

# Wait for a function to return truthy
page.wait_for_function('document.title === "Done"')

# Fixed delay (avoid if possible)
page.wait_for_timeout(2000)    # 2 seconds
```

Expect Navigation

```
# Click triggers a navigation – wrap with expect_navigation
with page.expect_navigation(url='**/success'):
    page.click('#submit')

# Wait for new page/tab to open
with context.expect_page() as page_info:
    page.click('a[target="_blank"]')
new_page = page_info.value
new_page.wait_for_load_state()
```

■■ **Warning:** Avoid `page.wait_for_timeout()` in tests — it makes tests slow and flaky. Use condition-based waits instead.

Custom Timeout

```
# Per-action timeout (ms)
page.click('#btn', timeout=10000)
page.wait_for_selector('.el', timeout=5000)

# Global default timeout
```

```
page.set_default_timeout(30000)      # 30 seconds  
page.set_default_navigation_timeout(60000)
```

7. Assertions

Playwright provides built-in async-aware assertions via `expect()`. These automatically retry until the condition is met or timeout.

Page Assertions

```
from playwright.sync_api import expect

# URL
expect(page).to_have_url('https://example.com')
expect(page).to_have_url(re.compile(r'/dashboard'))

# Title
expect(page).to_have_title('My App')
expect(page).to_have_title(re.compile(r'Dashboard'))
```

Locator Assertions

```
loc = page.locator('.message')

# Text content
expect(loc).to_have_text('Success!')
expect(loc).to_contain_text('Welcome')

# Visibility
expect(loc).to_be_visible()
expect(loc).to_be_hidden()

# Enabled/Disabled
expect(loc).to_be_enabled()
expect(loc).to_be_disabled()

# Checked state
expect(loc).to_be_checked()
expect(loc).not_to_be_checked()

# Attribute
expect(loc).to_have_attribute('href', '/home')

# CSS class
expect(loc).to_have_class('active')

# Count
expect(page.locator('li')).to_have_count(5)

# Input value
expect(page.get_by_label('Name')).to_have_value('Alice')

# Negation
expect(loc).not_to_be_visible()
```

■ **Tip:** `expect()` assertions auto-retry for up to 5 seconds by default. Adjust with `timeout` parameter.

8. Forms & File Uploads

Complete Form Example

```
# Fill a registration form
page.goto('https://example.com/register')

page.get_by_label('First Name').fill('John')
page.get_by_label('Last Name').fill('Doe')
page.get_by_label('Email').fill('john@example.com')
page.get_by_label('Password').fill('SecurePass123!')

# Select dropdown
page.get_by_label('Country').select_option('us')

# Check terms
page.get_by_label('I agree to terms').check()

# Submit
page.get_by_role('button', name='Register').click()

# Verify success
expect(page).to_have_url('**/welcome')
```

File Upload

```
# Single file upload
page.set_input_files('input[type="file"]', 'photo.jpg')

# Multiple files
page.set_input_files('input[type="file"]', [
  'doc1.pdf', 'doc2.pdf'
])

# Upload from buffer (no file on disk needed)
page.set_input_files('input[type="file"]', {
  'name': 'test.txt',
  'mimeType': 'text/plain',
  'buffer': b'Hello, World!'
})

# Intercept file chooser dialog
with page.expect_file_chooser() as fc_info:
  page.click('#upload-btn')
file_chooser = fc_info.value
file_chooser.set_files('image.png')
```

File Download

```
# Intercept download
with page.expect_download() as download_info:
  page.click('#download-btn')
download = download_info.value

# Save to specific path
```

```
download.save_as('/tmp/my_report.pdf')
```

```
# Get suggested filename  
print(download.suggested_filename)  
print(download.url)
```

9. Screenshots & PDFs

Screenshots

```
# Full page screenshot
page.screenshot(path='full_page.png', full_page=True)

# Viewport screenshot (default)
page.screenshot(path='viewport.png')

# Element screenshot
page.locator('#chart').screenshot(path='chart.png')

# Screenshot as bytes (no file)
img_bytes = page.screenshot()

# JPEG with quality
page.screenshot(path='page.jpg', type='jpeg', quality=85)

# Clip to region
page.screenshot(path='region.png', clip={
    'x': 0, 'y': 0, 'width': 800, 'height': 400
})

# Omit background (PNG only)
page.screenshot(path='transparent.png', omit_background=True)

# Mask elements
page.screenshot(
    path='masked.png',
    mask=[page.locator('.sensitive-data')]
)
```

PDF Generation

```
# Save page as PDF (Chromium only)
page.pdf(path='page.pdf')

# Custom paper size
page.pdf(
    path='custom.pdf',
    format='A4',
    margin={'top': '1cm', 'bottom': '1cm',
            'left': '1cm', 'right': '1cm'},
    print_background=True
)

# PDF as bytes
pdf_bytes = page.pdf()
```

10. Network Interception

Routing Requests

```
# Block all images
page.route('**/*.{png,jpg,gif,svg}',
    lambda route: route.abort())

# Block specific domains
page.route('**/analytics.js',
    lambda route: route.abort())

# Mock an API endpoint
page.route('**/api/users', lambda route: route.fulfill(
    status=200,
    content_type='application/json',
    body='{"users": [{"id": 1, "name": "Alice"}]}')
))

# Modify request
def modify_request(route):
    headers = {**route.request.headers,
               'X-Custom': 'test-value'}
    route.continue_(headers=headers)

page.route('**/*', modify_request)

# Modify response
def modify_response(route):
    response = route.fetch()
    body = response.json()
    body['extra'] = 'injected'
    route.fulfill(response=response, json=body)

page.route('**/api/data', modify_response)
```

Listening to Network Events

```
# Listen to all requests
page.on('request', lambda req: print(req.url, req.method))

# Listen to responses
page.on('response', lambda res: print(res.url, res.status))

# Wait for specific response
with page.expect_response('**/api/login') as resp_info:
    page.click('#login-btn')
response = resp_info.value
data = response.json()
print(data['token'])

# Wait for request
with page.expect_request('**/api/search') as req_info:
    page.fill('#search', 'playwright')
    page.press('#search', 'Enter')
```



```
request = req_info.value  
print(request.post_data)
```

11. JavaScript Execution

```
# Execute JS and get return value
title = page.evaluate('document.title')
count = page.evaluate('document.querySelectorAll("li").length')

# Pass arguments
result = page.evaluate('([a, b]) => a + b', [3, 7])
print(result)    # 10

# Multi-line function
scroll_pos = page.evaluate("""
    () => ({
        x: window.scrollX,
        y: window.scrollY
    })
""")

# evaluate vs evaluate_handle
# evaluate: returns serializable value (dict, list, str...)
# evaluate_handle: returns a JSHandle (reference to JS object)
handle = page.evaluate_handle('window.myApp')

# Execute on element
page.locator('#box').evaluate('el => el.style.border = "2px solid red"')

# Add script to page
page.add_script_tag(content='window.testFlag = true')
page.add_script_tag(url='https://cdn.example.com/lib.js')

# Add init script (runs before page loads)
page.add_init_script("window.__test__ = true")
context.add_init_script("window.__env__ = 'test'")
```

12. Frames & iFrames

```
# Get all frames
frames = page.frames
for f in frames:
    print(f.url, f.name)

# Get frame by name
frame = page.frame('frame-name')

# Get frame by URL
frame = page.frame(url='**/iframe-content')

# Interact with frame content
frame.click('#btn-inside-frame')
frame.fill('input', 'value')

# Frame locator (recommended approach)
frame_loc = page.frame_locator('iframe[name="payment"]')
frame_loc.get_by_label('Card Number').fill('4111111111111111')
frame_loc.get_by_role('button', name='Pay').click()

# Nested iframes
page.frame_locator('iframe#outer') \
    .frame_locator('iframe#inner') \
    .get_by_text('Submit').click()
```

■ **Tip:** `frame_locator()` is the preferred API — it integrates with locator chaining and auto-waiting.

13. Dialogs & Popups

Alert / Confirm / Prompt Dialogs

```
# Auto-accept all dialogs
page.on('dialog', lambda dialog: dialog.accept())

# Dismiss all dialogs
page.on('dialog', lambda dialog: dialog.dismiss())

# Handle specific dialog
def handle_dialog(dialog):
    print(f'Type: {dialog.type}')      # alert/confirm/prompt
    print(f'Message: {dialog.message}')
    if dialog.type == 'prompt':
        dialog.accept('my input')      # fill prompt then accept
    elif dialog.type == 'confirm':
        dialog.accept()                # click OK
    else:
        dialog.dismiss()               # click Cancel

page.on('dialog', handle_dialog)
page.click('#show-dialog')
```

New Tab / Popup Windows

```
# Catch a popup opened by a click
with context.expect_page() as popup_info:
    page.click('a[target="_blank"]')
popup = popup_info.value
popup.wait_for_load_state()
print(popup.title())
popup.close()

# Multiple tabs in same context
page1 = context.new_page()
page2 = context.new_page()
page1.goto('https://example.com')
page2.goto('https://other.com')
pages = context.pages      # list of all open pages
```

14. Authentication

Login Once, Reuse Session

```
# Step 1: Login and save state
def save_auth():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()
        page.goto('https://app.example.com/login')
        page.fill('#email', 'admin@example.com')
        page.fill('#password', 'secret')
        page.click('button[type="submit"]')
        page.wait_for_url('**/dashboard')
        # Save cookies + localStorage
        context.storage_state(path='auth.json')
        browser.close()

# Step 2: Use saved state in tests
context = browser.new_context(
    storage_state='auth.json'
)
page = context.new_page()
page.goto('https://app.example.com/dashboard')
# Already logged in!
```

HTTP Basic Auth

```
context = browser.new_context(
    http_credentials={
        'username': 'user',
        'password': 'pass'
    }
)
page = context.new_page()
page.goto('https://protected.example.com')
```

Cookie Management

```
# Add cookies
context.add_cookies([
    {
        'name': 'session_id',
        'value': 'abc123',
        'domain': 'example.com',
        'path': '/'
    }
])

# Get cookies
cookies = context.cookies()

# Clear cookies
context.clear_cookies()
```

15. Async API

The async API is ideal for high-performance scraping where you process many pages concurrently.

```
import asyncio
from playwright.async_api import async_playwright, expect

async def scrape_page(browser, url):
    page = await browser.new_page()
    await page.goto(url)
    title = await page.title()
    await page.close()
    return title

async def main():
    async with async_playwright() as p:
        browser = await p.chromium.launch()

        urls = [
            'https://example.com',
            'https://playwright.dev',
            'https://python.org',
        ]

        # Scrape all URLs concurrently
        tasks = [scrape_page(browser, url) for url in urls]
        titles = await asyncio.gather(*tasks)

        for url, title in zip(urls, titles):
            print(f'{url}: {title}')

        await browser.close()

asyncio.run(main())
```

■■ **Note:** Every async Playwright method must be awaited. Use `async_playwright()` instead of `sync_playwright()`.

16. pytest Integration

Installation

```
pip install pytest-playwright
playwright install
```

Basic Test

```
# test_login.py
from playwright.sync_api import Page, expect

def test_login_success(page: Page):
    page.goto('https://app.example.com/login')
    page.get_by_label('Email').fill('user@example.com')
    page.get_by_label('Password').fill('password123')
    page.get_by_role('button', name='Sign In').click()
    expect(page).to_have_url('**/dashboard')
    expect(page.get_by_text('Welcome back!')).to_be_visible()

def test_login_failure(page: Page):
    page.goto('https://app.example.com/login')
    page.get_by_label('Email').fill('wrong@example.com')
    page.get_by_label('Password').fill('wrongpassword')
    page.get_by_role('button', name='Sign In').click()
    expect(page.get_by_text('Invalid credentials')).to_be_visible()
```

Fixtures & confest.py

```
# confest.py
import pytest
from playwright.sync_api import Browser

@pytest.fixture(scope='session')
def auth_state(browser: Browser):
    context = browser.new_context()
    page = context.new_page()
    page.goto('https://app.example.com/login')
    page.fill('#email', 'admin@example.com')
    page.fill('#password', 'secret')
    page.click('#submit')
    page.wait_for_url('**/dashboard')
    state = context.storage_state()
    context.close()
    return state

@pytest.fixture
def auth_page(browser: Browser, auth_state):
    context = browser.new_context(storage_state=auth_state)
    page = context.new_page()
    yield page
    context.close()

# test_dashboard.py
def test_dashboard_loads(auth_page): # already logged in!
```

```
auth_page.goto('https://app.example.com/dashboard')
expect(auth_page.get_by_heading('Dashboard')).to_be_visible()
```

Running Tests

```
# Run all tests (headless Chromium)
pytest

# Headed mode
pytest --headed

# Specific browser
pytest --browser firefox
pytest --browser webkit

# All browsers
pytest --browser chromium --browser firefox --browser webkit

# Slow motion
pytest --slowmo 500

# Parallel (requires pytest-xdist)
pytest -n auto

# Save traces on failure
pytest --tracing=retain-on-failure

# HTML report
pytest --html=report.html
```

pytest.ini / pyproject.toml

```
# pyproject.toml
[tool.pytest.ini_options]
addopts = "--headed --browser chromium --tracing=retain-on-failure"

# pytest.ini
[pytest]
addopts = --browser=chromium
```


17. Page Object Model (POM)

The Page Object Model organises selectors and actions into reusable classes, making tests cleaner and easier to maintain.

```
# pages/login_page.py
from playwright.sync_api import Page, expect

class LoginPage:
    def __init__(self, page: Page):
        self.page = page
        self.email = page.get_by_label('Email')
        self.password = page.get_by_label('Password')
        self.submit = page.get_by_role('button', name='Sign In')
        self.error = page.get_by_role('alert')

    def goto(self):
        self.page.goto('/login')
        return self

    def login(self, email: str, password: str):
        self.email.fill(email)
        self.password.fill(password)
        self.submit.click()
        return self

    def assert_error(self, message: str):
        expect(self.error).to_contain_text(message)
        return self

# pages/dashboard_page.py
class DashboardPage:
    def __init__(self, page: Page):
        self.page = page
        self.heading = page.get_by_role('heading', name='Dashboard')

    def assert_loaded(self):
        expect(self.heading).to_be_visible()
        return self

# test_login.py
def test_login(page):
    login = LoginPage(page).goto()
    login.login('admin@example.com', 'secret')
    DashboardPage(page).assert_loaded()

def test_bad_login(page):
    LoginPage(page).goto() \
        .login('wrong@test.com', 'bad') \
        .assert_error('Invalid credentials')
```

18. Debugging & Tracing

Playwright Inspector

```
# Open inspector (pause execution)
page.pause()

# Run with PWDEBUG env var
# PWDEBUG=1 python script.py

# Codegen - record browser actions as code
# playwright codegen https://example.com
# playwright codegen --target=python https://example.com
```

Traces

```
# Record a trace
context.tracing.start(
    screenshots=True,
    snapshots=True,
    sources=True
)

# ... run your test ...

context.tracing.stop(path='trace.zip')

# View trace
# playwright show-trace trace.zip
```

Video Recording

```
context = browser.new_context(
    record_video_dir='videos/',
    record_video_size={'width': 1280, 'height': 720}
)
page = context.new_page()
# ... test ...
context.close() # video saved here
print(page.video.path())
```

Console & Error Logging

```
# Listen to console messages
page.on('console', lambda msg: print(f'[{msg.type}] {msg.text}'))

# Listen to page errors
page.on('pageerror', lambda err: print(f'Error: {err}'))

# Listen to requests failing
page.on('requestfailed', lambda req:
    print(f'FAILED: {req.url} - {req.failure()}'))
```

19. Mobile Emulation

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    # Use a pre-defined device
    iphone = p.devices['iPhone 13']
    pixel = p.devices['Pixel 5']

    context = p.chromium.launch().new_context(**iphone)
    page = context.new_page()
    page.goto('https://example.com')
    page.screenshot(path='mobile.png')

    # Custom device
    context2 = p.chromium.launch().new_context(
        viewport={'width': 375, 'height': 812},
        device_scale_factor=3,
        is_mobile=True,
        has_touch=True,
        user_agent=(
            'Mozilla/5.0 (iPhone; CPU iPhone OS 15_0 like Mac OS X) '
            'AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148'
        )
    )

# Available devices include:
# 'iPhone 13', 'iPhone 13 Pro Max', 'iPad Pro'
# 'Pixel 5', 'Galaxy S9+', 'Galaxy Tab S4'
# Use p.devices to list all available
```

20. Best Practices

Selector Priority (best to worst)

Priority	Selector	Example
1 (Best)	<code>get_by_role()</code>	<code>get_by_role('button', name='Submit')</code>
2	<code>get_by_label()</code>	<code>get_by_label('Email address')</code>
3	<code>get_by_test_id()</code>	<code>get_by_test_id('login-form')</code>
4	<code>get_by_text()</code>	<code>get_by_text('Sign In')</code>
5	CSS selector	<code>locator('.btn-primary')</code>
6 (Worst)	XPath	<code>locator('xpath=//button')</code>

Do's & Don'ts

Do:

- ✓ Use locators instead of `page.click(selector)` directly
- ✓ Prefer semantic selectors (role, label) over CSS/XPath
- ✓ Use `expect()` assertions — they auto-retry
- ✓ Store auth state in a file and reuse it
- ✓ Use Page Object Model for large test suites
- ✓ Set reasonable default timeouts
- ✓ Run tests in parallel with `pytest-xdist`
- ✓ Use tracing to debug flaky tests

Don't:

- ✗ Don't use `page.wait_for_timeout()` — it's fragile
- ✗ Don't rely on fixed delays; use event-based waits
- ✗ Don't use overly brittle CSS selectors tied to styling
- ✗ Don't share browser/context state between tests
- ✗ Don't ignore console errors during testing
- ✗ Don't skip closing contexts/browsers

Quick Reference: Key Methods

Category	Method	Description
Navigate	<code>page.goto(url)</code>	Navigate to URL
Find	<code>page.locator(sel)</code>	Find element(s)

Find	<code>page.get_by_role()</code>	Find by ARIA role
Action	<code>loc.click()</code>	Click element
Action	<code>loc.fill(text)</code>	Fill input
Action	<code>loc.press(key)</code>	Press keyboard key
Assert	<code>expect(loc).to_be_visible()</code>	Assert visibility
Assert	<code>expect(page).to_have_url()</code>	Assert URL
Network	<code>page.route(pattern, fn)</code>	Intercept requests
JS	<code>page.evaluate(expr)</code>	Run JavaScript
Debug	<code>page.pause()</code>	Open inspector
Auth	<code>context.storage_state()</code>	Save session